

Go Code Challenge 3

Сервис переводов между аккаунтами

ℹ️ Введение

Наша компания разрабатывает онлайн-игру с внутриигровой валютой ("монеты"). Игроки могут переводить монеты друг другу — для обмена предметами или подарков друзьям.

Необходимо реализовать API-сервис, обрабатывающий переводы монет между аккаунтами игроков.

Сценарий

1. Система запускается с предзаполненными аккаунтами игроков, каждый из которых имеет известный начальный баланс.
2. Игроки инициируют переводы через API. Каждый запрос на перевод содержит ключ идемпотентности (предоставляется клиентом) для защиты от сетевых ретраев и дублирующих запросов.

📡 API-контракт

Перевод монет:

```
POST /api/v1/transfers
Content-Type: application/json

Запрос:
{
  "idempotency_key": "550e8400-e29b-41d4-a716-446655440000",
  "from_account_id": "acc_1",
  "to_account_id": "acc_2",
  "amount": 100
}

Успешный перевод:
```

```
HTTP 200
{
  "transfer_id": "...",
  "status": "completed",
  "from_account_id": "acc_1",
  "to_account_id": "acc_2",
  "amount": 100
}

Отклонённый перевод (например, недостаточно средств):
HTTP 422
{
  "status": "rejected",
  "reason": "insufficient funds"
}
```

Получение баланса аккаунта:

```
GET /api/v1/accounts/{id}

HTTP 200
{
  "account_id": "acc_1",
  "balance": 900
}

HTTP 404 — аккаунт не найден
```

Получение перевода по ключу идемпотентности:

```
GET /api/v1/transfers/{idempotency_key}

HTTP 200
{
  "transfer_id": "...",
  "idempotency_key": "550e8400-e29b-41d4-a716-446655440000",
  "from_account_id": "acc_1",
  "to_account_id": "acc_2",
  "amount": 100,
  "status": "completed",
  "created_at": "2025-01-01T00:00:00Z"
}

HTTP 404 — перевод не найден
```

📋 Бизнес-правила

1. **Ограничение баланса**: баланс аккаунта никогда не может стать отрицательным. Перевод, который привёл бы к отрицательному балансу, должен быть отклонён со статусом `"rejected"` и причиной `"insufficient funds"`.
2. **Идемпотентность**:
   - Повторный запрос с тем же `idempotency_key` должен вернуть тот же ответ, что и оригинальный запрос, и **не** должен применять перевод повторно.
   - Запрос с существующим `idempotency_key`, но **другими** данными (`from_account_id`, `to_account_id` или `amount`) должен быть отклонён с HTTP `409 Conflict`.
3. **Перевод самому себе**: перевод, где `from_account_id == to_account_id`, должен быть отклонён с HTTP `422` и причиной `"self-transfer not allowed"`.
4. **Несуществующий аккаунт**: если один из аккаунтов не существует, вернуть HTTP `422` с причиной `"account not found"`.
5. **Валидация**: `idempotency_key`, `from_account_id`, `to_account_id` должны быть непустыми строками. Ошибки валидации возвращают HTTP `400`.

🔧 Технические требования

- **Язык**: Go
- **База данных**: на ваш выбор (PostgreSQL, MongoDB и т.д.)
- **Запуск**: предоставьте `docker-compose.yml`, который поднимает сервис, базу данных и выполняет начальное заполнение данных
- **Порт**: сервис должен быть доступен по адресу `http://localhost:3000/`
- **Начальные данные**: при запуске должны быть созданы следующие аккаунты:

ID аккаунта	Начальный баланс
`acc_1`	1000
`acc_2`	1000

ID аккаунта	Начальный баланс
`acc_3`	1000
`acc_4`	1000
`acc_5`	1000
`acc_6`	1000
`acc_7`	1000
`acc_8`	1000
`acc_9`	1000
`acc_10`	1000

Общий баланс системы при старте: **10 000 монет**.

🧪 Тесты

Напишите тесты. Полное покрытие не требуется, но продемонстрируйте свой подход к тестированию на значимых примерах. Сосредоточьтесь на тех кейсах, которые считаете наиболее важными для сервиса такого типа.

📝 Сопроводительное описание

Так как в рамках тестового задания практически невозможно довести проект до production-ready состояния, пожалуйста, приложите краткое описание, включающее:

- Что бы вы добавили или изменили для продакшена.
- Почему выбрали именно эту базу данных, фреймворк и библиотеки.
- Ваши мысли об инфраструктуре, деплое и масштабировании этого сервиса.
- Какие компромиссы вы приняли и почему.

🚀 Публикация решения

Решение необходимо залить в **приватный** репозиторий на GitHub и предоставить доступ пользователям `@volkovchain`, `@Powerz`.